FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

LABEXER1

Charles Gabriel Marcelo	Doc. Hadji Tejuco
Name of Student	Name of Professor
October 16, 2026	October 16, 2026
Data Performed	Date Submitted

Program Overview:

This program is a 32-bit NASM assembly calculator for Linux that allows users to perform addition, subtraction, multiplication, and division. It displays a menu where the user selects an operation using A, S, M, or D, then asks for two numbers between 0 and 99. The program reads and validates the input, performs the selected arithmetic operation, and displays the result. For division, it shows both the quotient and remainder while preventing division by zero. After displaying the result, the program asks whether the user wants to try again; entering Y returns to the menu, while entering N terminates the program. The updated version also fixes the input bug by reading the numbers correctly after displaying each prompt and storing the selected operation safely in memory.

Paste the source code and explain the functionalities and take a screenshot of the final output.

GITHUB:
https://github.com/kizern/Marcelo_TN24_LABEXER1

SOURCE CODE:

```
global _start

%define SYS_EXIT  1
%define SYS_READ  3
%define SYS_WRITE 4
%define STDIN     0
%define STDOUT    1

section .data
   title      db "Calculator", 10
   title_len   equ $-title
   menu       db "-----------------------------", 10
              db "MENU", 10
              db "[A]dd", 10
              db "[S]ubtract", 10
              db "[M]ultiply", 10
              db "[D]ivide", 10
              db "-----------------------------", 10
   menu_len    equ $-menu
   choice_msg  db "Enter your choice: "
   choice_len  equ $-choice_msg
   first_msg   db "Enter first number (0-99): "
   first_len   equ $-first_msg
   second_msg  db "Enter second number (0-99): "
   second_len  equ $-second_msg
```

```
    result_msg  db "The result is "
    result_len  equ $-result_msg
    quotient_msg db "The quotient is "
    quotient_len equ $-quotient_msg
    remainder_msg db "The remainder is "
    remainder_len equ $-remainder_msg
    retry_msg   db 10, "Do you want to try again (Y/N)? "
    retry_len   equ $-retry_msg
    invalid_choice db "Invalid choice. Please enter A, S, M, or D.", 10
    invalid_choice_len equ $-invalid_choice
    invalid_number db "Invalid number. Enter a value from 0 to 99.", 10
    invalid_number_len equ $-invalid_number
    divide_zero db "Cannot divide by zero.", 10
    divide_zero_len equ $-divide_zero
    newline     db 10
    newline_len equ $-newline
    clear       db 27, "[2J", 27, "[H"
    clear_len   equ $-clear

section .bss
    input       resb 64
    number_buf  resb 16
    first_num   resd 1
    second_num  resd 1
    result      resd 1
    operation   resb 1

section .text

_start:
main_loop:
    call clear_screen
    call print_title_menu

    mov ecx, choice_msg
    mov edx, choice_len
    call write_string
    call read_line
    call first_non_space
    cmp al, 'A'
    je .operation_selected
    cmp al, 'a'
    je .operation_selected
```

```
    cmp al, 'S'
    je .operation_selected
    cmp al, 's'
    je .operation_selected
    cmp al, 'M'
    je .operation_selected
    cmp al, 'm'
    je .operation_selected
    cmp al, 'D'
    je .operation_selected
    cmp al, 'd'
    je .operation_selected
    mov ecx, invalid_choice
    mov edx, invalid_choice_len
    call write_string
    jmp ask_retry

.operation_selected:
    mov [operation], al

    mov ecx, first_msg
    mov edx, first_len
    call write_string
    call read_line
    call read_number
    jc .bad_number
    mov [first_num], eax

    mov ecx, second_msg
    mov edx, second_len
    call write_string
    call read_line
    call read_number
    jc .bad_number
    mov [second_num], eax

    cmp byte [operation], 'A'
    je .add
    cmp byte [operation], 'a'
    je .add
    cmp byte [operation], 'S'
    je .subtract
    cmp byte [operation], 's'
```

```
    je .subtract
    cmp byte [operation], 'M'
    je .multiply
    cmp byte [operation], 'm'
    je .multiply
    cmp byte [operation], 'D'
    je .divide
    ; lowercase d
    jmp .divide

.add:
    mov eax, [first_num]
    add eax, [second_num]
    mov [result], eax
    call print_result
    jmp ask_retry

.subtract:
    mov eax, [first_num]
    sub eax, [second_num]
    mov [result], eax
    call print_result
    jmp ask_retry

.multiply:
    mov eax, [first_num]
    imul eax, [second_num]
    mov [result], eax
    call print_result
    jmp ask_retry

.divide:
    cmp dword [second_num], 0
    je .zero_divisor
    mov eax, [first_num]
    xor edx, edx
    div dword [second_num]
    push edx
    mov [result], eax
    mov ecx, quotient_msg
    mov edx, quotient_len
    call write_string
    mov eax, [result]
```

```
    call print_integer
    mov ecx, newline
    mov edx, newline_len
    call write_string
    pop eax
    mov [result], eax
    mov ecx, remainder_msg
    mov edx, remainder_len
    call write_string
    mov eax, [result]
    call print_integer
    mov ecx, newline
    mov edx, newline_len
    call write_string
    jmp ask_retry

.zero_divisor:
    mov ecx, divide_zero
    mov edx, divide_zero_len
    call write_string
    jmp ask_retry

.bad_number:
    mov ecx, invalid_number
    mov edx, invalid_number_len
    call write_string

ask_retry:
    mov ecx, retry_msg
    mov edx, retry_len
    call write_string
    call read_line
    call first_non_space
    cmp al, 'Y'
    je main_loop
    cmp al, 'y'
    je main_loop
    call exit_program

read_line:
    mov eax, SYS_READ
    mov ebx, STDIN
    mov ecx, input
```

```
    mov edx, 63
    int 80h
    cmp eax, 0
    jle exit_program
    mov byte [input+eax], 0
    ret

first_non_space:
    mov esi, input
.scan:
    mov al, [esi]
    cmp al, ' '
    je .next
    cmp al, 9
    je .next
    ret
.next:
    inc esi
    jmp .scan

read_number:
    push ebx
    push esi
    push edi
    mov esi, input
    xor eax, eax
    xor ebx, ebx
.skip:
    mov dl, [esi]
    cmp dl, ' '
    je .advance
    cmp dl, 9
    je .advance
    jmp .digits
.advance:
    inc esi
    jmp .skip
.digits:
    mov dl, [esi]
    cmp dl, 10
    je .finish
    cmp dl, 13
    je .finish
```

```
    cmp dl, 0
    je .finish
    cmp dl, '0'
    jb .invalid
    cmp dl, '9'
    ja .invalid
    imul eax, 10
    sub dl, '0'
    movzx edi, dl
    add eax, edi
    cmp eax, 99
    ja .invalid
    inc ebx
    inc esi
    jmp .digits
.finish:
    cmp ebx, 0
    je .invalid
    clc
    pop edi
    pop esi
    pop ebx
    ret
.invalid:
    stc
    pop edi
    pop esi
    pop ebx
    ret

print_result:
    mov ecx, result_msg
    mov edx, result_len
    call write_string
    mov eax, [result]
    call print_integer
    mov ecx, newline
    mov edx, newline_len
    call write_string
    ret

print_integer:
    push eax
```

```
    test eax, eax
    jns .positive
    mov ecx, minus
    mov edx, 1
    call write_string
    pop eax
    neg eax
    jmp .convert
.positive:
    pop eax
.convert:
    mov edi, number_buf
    add edi, 15
    mov byte [edi], 0
    mov ebx, 10
    cmp eax, 0
    jne .digits
    dec edi
    mov byte [edi], '0'
    jmp .output
.digits:
    xor edx, edx
    div ebx
    add dl, '0'
    dec edi
    mov [edi], dl
    test eax, eax
    jnz .digits
.output:
    mov ecx, edi
    mov edx, number_buf
    add edx, 15
    sub edx, edi
    call write_string
    ret

minus db '-'

print_title_menu:
    mov ecx, title
    mov edx, title_len
    call write_string
    mov ecx, menu
```

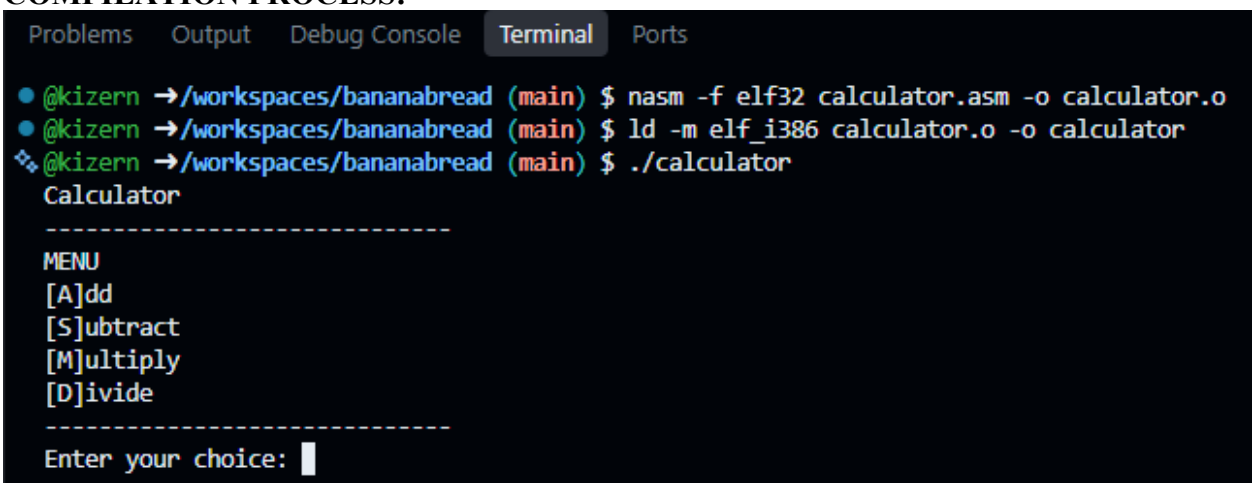
```
    mov edx, menu_len
    call write_string
    ret

clear_screen:
    mov ecx, clear
    mov edx, clear_len
    call write_string
    ret

write_string:
    push eax
    push ebx
    mov eax, SYS_WRITE
    mov ebx, STDOUT
    int 80h
    pop ebx
    pop eax
    ret

exit_program:
    mov eax, SYS_EXIT
    xor ebx, ebx
    int 80h
```

COMPILATION PROCESS:



TEST CASE – Addition:

#1:

```
------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice: A
Enter first number (0-99): 99
Enter second number (0-99): 10
The result is 109
```

#2:

```
@kizern ➜/workspaces/bananabread (main) $ ./calculator
------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice: A
Enter first number (0-99): -10
Invalid number. Enter a value from 0 to 99.
```

#3:

```
Calculator
------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
------------------------------
Enter your choice: A
Enter first number (0-99): 1000
Invalid number. Enter a value from 0 to 99.

Do you want to try again (Y/N)? n
```

TEST CASE – Subtraction:

#1:

```
@kizern →/workspaces/bananabread (main) $ nasm -f elf32 calculator.asm -o calculator.o
@kizern →/workspaces/bananabread (main) $ ld -m elf_i386 calculator.o -o calculator
@kizern →/workspaces/bananabread (main) $ ./calculator
Calculator
-----------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
-----------------------------
Enter your choice: S
Enter first number (0-99): -10
Invalid number. Enter a value from 0 to 99.

Do you want to try again (Y/N)? n
```

#2:

```
@kizern →/workspaces/bananabread (main) $ nasm -f elf32 calculator.asm -o calculator.o
@kizern →/workspaces/bananabread (main) $ ld -m elf_i386 calculator.o -o calculator
@kizern →/workspaces/bananabread (main) $ ./calculator
Calculator
-----------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
-----------------------------
Enter your choice: S
Enter first number (0-99): 50
Enter second number (0-99): 60
The result is -10

Do you want to try again (Y/N)? n
```

#3:

```
@kizern →/workspaces/bananabread (main) $ nasm -f elf32 calculator.asm -o calculator.o
@kizern →/workspaces/bananabread (main) $ ld -m elf_i386 calculator.o -o calculator
@kizern →/workspaces/bananabread (main) $ ./calculator
Calculator
----------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your choice: S
Enter first number (0-99): 1000
Invalid number. Enter a value from 0 to 99.

Do you want to try again (Y/N)? n
@kizern →/workspaces/bananabread (main) $
```

TEST CASE – Multiplication:

#1:

```
@kizern →/workspaces/bananabread (main) $ nasm -f elf32 calculator.asm -o calculator.o
@kizern →/workspaces/bananabread (main) $ ld -m elf_i386 calculator.o -o calculator
@kizern →/workspaces/bananabread (main) $ ./calculator
Calculator
----------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your choice: M
Enter first number (0-99): 10
Enter second number (0-99): 5
The result is 50

Do you want to try again (Y/N)? n
@kizern →/workspaces/bananabread (main) $
```

#2:

```
@kizern →/workspaces/bananabread (main) $ nasm -f elf32 calculator.asm -o calculator.o
@kizern →/workspaces/bananabread (main) $ ld -m elf_i386 calculator.o -o calculator
@kizern →/workspaces/bananabread (main) $ ./calculator
Calculator
----------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your choice: m
Enter first number (0-99): -10
Invalid number. Enter a value from 0 to 99.

Do you want to try again (Y/N)? n
@kizern →/workspaces/bananabread (main) $
```

#3:

```
@kizern →/workspaces/bananabread (main) $ nasm -f elf32 calculator.asm -o calculator.o
@kizern →/workspaces/bananabread (main) $ ld -m elf_i386 calculator.o -o calculator
@kizern →/workspaces/bananabread (main) $ ./calculator
Calculator
----------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your choice: M
Enter first number (0-99): 1000
Invalid number. Enter a value from 0 to 99.

Do you want to try again (Y/N)? n
@kizern →/workspaces/bananabread (main) $
```

TEST CASE – Division:

#1:

```
@kizern →/workspaces/bananabread (main) $ nasm -f elf32 calculator.asm -o calculator.o
@kizern →/workspaces/bananabread (main) $ ld -m elf_i386 calculator.o -o calculator
@kizern →/workspaces/bananabread (main) $ ./calculator
Calculator
----------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your choice: D
Enter first number (0-99): 50
Enter second number (0-99): 5
The quotient is 10
The remainder is 0

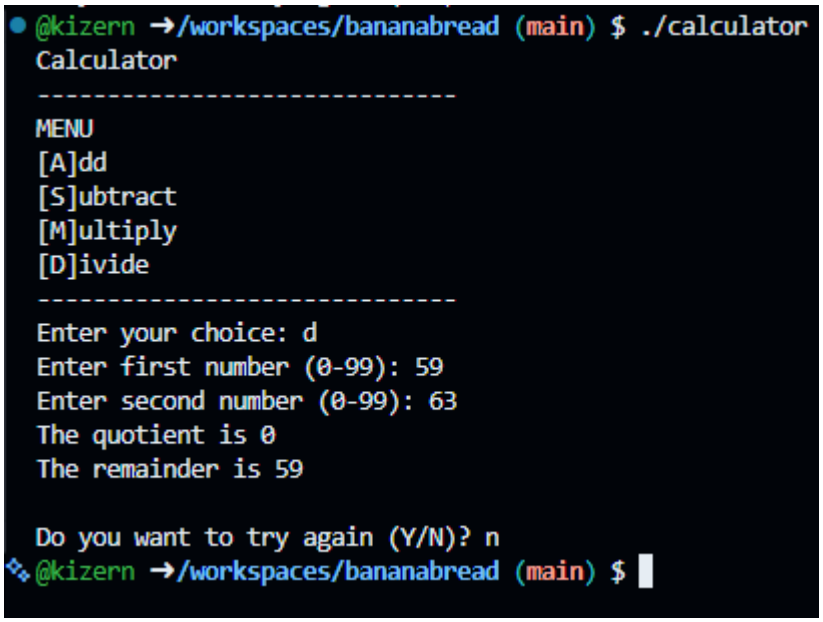
Do you want to try again (Y/N)? n
@kizern →/workspaces/bananabread (main) $
```

#2:

```
@kizern →/workspaces/bananabread (main) $ nasm -f elf32 calculator.asm -o calculator.o
@kizern →/workspaces/bananabread (main) $ ld -m elf_i386 calculator.o -o calculator
@kizern →/workspaces/bananabread (main) $ ./calculator
Calculator
----------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------
Enter your choice: D
Enter first number (0-99): -10
Invalid number. Enter a value from 0 to 99.

Do you want to try again (Y/N)? n
@kizern →/workspaces/bananabread (main) $
```

#3:


@kizern →/workspaces/bananabread (main) $ ./calculator
Calculator
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
Enter your choice: d
Enter first number (0-99): 59
Enter second number (0-99): 63
The quotient is 0
The remainder is 59
Do you want to try again (Y/N)? n
@kizern →/workspaces/bananabread (main) $